



Yıldız Teknik Üniversitesi
Elektrik-Elektronik Fakültesi
Bilgisayar Mühendisliği Bölümü

BLM1031

Yapısal Programlama

Gr 2

Öğr. Gör. Dr. Ahmet ELBİR

Dönem Projesi

İsim: Alperen Ateş

No: 25011018

E-Posta:

Adım adım ekran görüntüleri

```
+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+
Enter main menu operation [1-4]: |
```

Ana menü

```
Enter main menu operation [1-4]: 1
Enter matrix size [3-10]: 4
Enter Player Name: Alperen
```

```
Creating random 4x4 matrix . . .
Matrix created
```

```
  | 0 | 1 | 2 | 3 |
-----
0 | 2 | 4 | 1 |
-----
1 |  |  | 1 |  |
-----
2 | 2 |  |  |  |
-----
3 | 4 |  | 3 | 3 |
-----
```

```
+-----+
| 1- Play in Automatic Mode |
| 2- Play in Manual Mode |
| 3- Return to Main Menu |
+-----+
```

```
Enter game menu operation [1-3]: |
```

4x4'lük rastgele matris oluşturulur

```

+-----+
| 1- Play in Automatic Mode |
| 2- Play in Manual Mode   |
| 3- Return to Main Menu    |
+-----+
Enter game menu operation [1-3]: 2

  | 0| 1| 2| 3|
-----
0|| 2|  | 4| 1|
-----
1||  |  | 1|  |
-----
2|| 2|  |  |  |
-----
3|| 4|  | 3| 3|
-----

+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: |

```

Manuel çözme modunu seçilir

```

  | 0| 1| 2| 3|
-----
0|| 2|  | 4| 1|
-----
1|| 2|  | 1|  |
-----
2|| 2|  |  |  |
-----
3|| 4|  | 3| 3|
-----

+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 3 0
Enter destination position (row, column): 3 1

  | 0| 1| 2| 3|
-----
0|| 2|  | 4| 1|
-----
1|| 2|  | 1|  |
-----
2|| 2|  |  |  |
-----
3|| 4| 4| 3| 3|
-----

+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 3 1
Enter destination position (row, column): 0 1

  | 0| 1| 2| 3|
-----
0|| 2| 4| 4| 1|
-----
1|| 2| 4| 1|  |
-----
2|| 2| 4|  |  |
-----
3|| 4| 4| 3| 3|
-----

+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: |

```

Oyunda biraz ilerlenir. Ardından oyun kapatılır

```

      | 0| 1| 2| 3|
-----
0|| 2| 4| 4| 1|
-----
1|| 2| 4| 1|  |
-----
2|| 2| 4|  |  |
-----
3|| 4| 4| 3| 3|
-----
+-----+
| 1- Undo |
| 2- Move |
| 3- Resign|
+-----+
Enter operation [1-3]: |

```

Oyun geri açıldığında doğrudan son oyun açılır

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign|
+-----+
Enter operation [1-3]: 1

      | 0| 1| 2| 3|
-----
0|| 2|  | 4| 1|
-----
1|| 2|  | 1|  |
-----
2|| 2|  |  |  |
-----
3|| 4| 4| 3| 3|
-----
+-----+
| 1- Undo |
| 2- Move |
| 3- Resign|
+-----+
Enter operation [1-3]: |

```

Kayıttan yükleme yapıldığında geri alma mekanizmasının çalışması bozulmaz

```

+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 3 1
Enter destination position (row, column): 0 1

| 0| 1| 2| 3|
+-----+
0|| 2| 4| 4| 1|
+-----+
1|| 2| 4| 1|  |
+-----+
2|| 2| 4|  |  |
+-----+
3|| 4| 4| 3| 3|
+-----+
| 1- Undo   |
| 2- Move   |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 3 3
Enter destination position (row, column): 2 3

| 0| 1| 2| 3|
+-----+
0|| 2| 4| 4| 1|
+-----+
1|| 2| 4| 1| 1|
+-----+
2|| 2| 4| 3| 3|
+-----+
3|| 4| 4| 3| 3|
+-----+
Time (second) elapsed: 330
Undo count: 1
Score: 1898.771070
+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+

```

Oyun manuel olarak çözülür ve ana menüye geri gelinir

```

+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+
Enter main menu operation [1-4]: 2
Enter matrix size [3-10]: 5
Enter Player Name: Ates
Enter 5x5 matrix file name: ornek.txt

  | 0 | 1 | 2 | 3 | 4 |
+-----+
0 | | | | 2 | 5 |
+-----+
1 | | 1 | | | |
+-----+
2 | | | | 4 | 3 |
+-----+
3 | 2 | 1 | 5 | | |
+-----+
4 | 4 | | | | 3 |
+-----+
+-----+
| 1- Play in Automatic Mode |
| 2- Play in Manual Mode |
| 3- Return to Main Menu |
+-----+
Enter game menu operation [1-3]: |

```

Dosyadan 5x5 boyutunda bir matris yüklenir. (yüklenen dosyanın her satırında “satır sütun değer” şeklinde veriler text biçiminde tutulmalıdır)

7

```

undoing 1 from (0, 0)
undoing 1 from (1, 0)
undoing 1 from (2, 0)
moving number 1 from (2, 1) to (2, 2)
moving number 1 from (2, 2) to (1, 2)
moving number 1 from (1, 2) to (1, 3)
moving number 1 from (1, 3) to (1, 4)
undoing 1 from (1, 4)
undoing 1 from (1, 3)
moving number 1 from (1, 2) to (0, 2)
moving number 1 from (0, 2) to (0, 1)
moving number 1 from (0, 1) to (0, 0)
moving number 1 from (0, 0) to (1, 0)
moving number 1 from (1, 0) to (2, 0)
undoing 1 from (2, 0)
undoing 1 from (1, 0)
undoing 1 from (0, 0)
undoing 1 from (0, 1)
undoing 1 from (0, 2)
undoing 1 from (1, 2)
undoing 1 from (2, 2)
moving number 1 from (2, 1) to (3, 1)
moving number 2 from (0, 3) to (0, 2)
moving number 2 from (0, 2) to (0, 1)
moving number 2 from (0, 1) to (0, 0)
moving number 2 from (0, 0) to (1, 0)
moving number 2 from (1, 0) to (2, 0)
moving number 2 from (2, 0) to (3, 0)
moving number 3 from (2, 4) to (1, 4)
moving number 3 from (1, 4) to (1, 3)
moving number 3 from (1, 3) to (1, 2)
moving number 3 from (1, 2) to (2, 2)
undoing 3 from (2, 2)
undoing 3 from (1, 2)
undoing 3 from (1, 3)
undoing 3 from (1, 4)
moving number 3 from (2, 4) to (3, 4)
moving number 3 from (3, 4) to (3, 3)
moving number 3 from (3, 3) to (4, 3)
moving number 3 from (4, 3) to (4, 2)
moving number 3 from (4, 2) to (4, 1)
undoing 3 from (4, 1)
undoing 3 from (4, 2)
moving number 3 from (4, 3) to (4, 4)
moving number 4 from (2, 3) to (2, 2)
moving number 4 from (2, 2) to (1, 2)
moving number 4 from (1, 2) to (1, 3)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 4 from (1, 2)
undoing 4 from (2, 2)
moving number 4 from (2, 3) to (1, 3)
moving number 4 from (1, 3) to (1, 2)
undoing 4 from (1, 2)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 3 from (4, 4)
undoing 3 from (4, 3)
undoing 3 from (3, 3)
moving number 3 from (3, 4) to (4, 4)
moving number 4 from (2, 3) to (2, 2)
moving number 4 from (2, 2) to (1, 2)
moving number 4 from (1, 2) to (1, 3)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 4 from (1, 2)
undoing 4 from (2, 2)
moving number 4 from (2, 3) to (1, 3)
moving number 4 from (1, 3) to (1, 2)
undoing 4 from (1, 2)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
moving number 4 from (2, 3) to (3, 3)
moving number 4 from (3, 3) to (4, 3)
moving number 4 from (4, 3) to (4, 2)
moving number 4 from (4, 2) to (4, 1)
moving number 4 from (4, 1) to (4, 0)
moving number 5 from (0, 4) to (1, 4)

```

```

moving number 5 from (1, 4) to (1, 3)
moving number 5 from (1, 3) to (1, 2)
moving number 5 from (1, 2) to (2, 2)
moving number 5 from (2, 2) to (3, 2)

```

```

| 0| 1| 2| 3| 4|
-----
0|| 2| 2| 2| 2| 5|
-----
1|| 2| 1| 5| 5| 5|
-----
2|| 2| 1| 5| 4| 3|
-----
3|| 2| 1| 5| 4| 3|
-----
4|| 4| 4| 4| 4| 3|
-----

```

```

Time (second) elapsed: 0
Undo count: 112
Score: 650.000000

```

```

+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+

```

```

Enter main menu operation [1-4]: |

```


Dosyadan okunan bu matris otomatik modda çözülür ve ana menüye dönülür.

```
+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+
Enter main menu operation [1-4]: 3
Name: Alperen Score: 1898.771070
Name: Ates Score: 650.000000
Press Enter (Return) key to continue . . .

+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+
Enter main menu operation [1-4]: |
```

Puanlar listelenir

Otomatik Modda Yol Bulma

Yol bulma algoritması ve açıklamaları

game_menu_automatic

```
/* Input: ListOfLists *lst.    Output: bool (success/fail), ListOfLists *lst */
/* automatic gameplay menu. return true if succeed */
bool game_menu_automatic(ListOfLists *lst)
{
    Score *s = NULL;

    CHECK_LISTOFLISTS(lst, return false);

    if(lst->g.matrixCount == 0)
    {
        fprintf(stderr, "There is no matrix to solve\n");
        return false;
    }

    #if (VERBOSE_LEVEL >= 1)
    if(lst->g.matrixSize >= MATRIX_WARNING_THRESHOLD)
    {
        fprintf(stderr, "This may take long time to solve\n");
    }
    #endif

    lst->g.totalTime = (lst->g.totalTime == 0 ? (unsigned long)time(NULL) : lst->g.totalTime);

    if(gamestate_solver(&lst->g) == false)
    {
        fprintf(stderr, "Matrix is not solveable\n");
        return false;
    }

    lst->g.totalTime = time(NULL) - lst->g.totalTime;

    matrix_print(&lst->g.mat[lst->g.matrixCount-1]);

    printf("Time (second) elapsed: %lu\n", lst->g.totalTime);
    printf("Undo count: %lu\n", lst->g.undoCount);

    s = score_create(lst->g.playerName, lol_calculate_score(lst));
    scorelist_add_in_order(&lst->slist, s);
    scorelist_file_write(&lst->slist);
    printf("Score: %lf\n", s->score);

    remove(FILENAME_LAST_GAME);

    return true;
}
```

Matris dosyadan alındıktan veya rastgele oluşturulduktan sonraki menüde otomatik çözme seçildiğinde game_menu_automatic fonksiyonu çağırılmaktadır. Fonksiyonun adım adım açıklaması:

1. Önce CHECK_LISTOFLISTS makrosu ile gelen parametrenin doğruluğu ve hemen ardından GameState içinde herhangi bir matris olup olmadığı kontrol edilir.
2. Matrisin boyutu belirlenen bir eşikten (MATRIX_WARNING_THRESHOLD) büyük veya eşit olduğu durumda, işlemin uzun sürebileceği kullanıcıya bildirilir.

3. Oyunun başlangıcından sona kadar geçen süreyi ölçebilmek için işleme başlamadan eğer oyun bir kayıttan yüklenmediyse o anki zaman time fonksiyonu ile alınır.
4. gamestate_solver fonksiyonu ile matris çözülür (aşağıda açıklanmıştır).
5. Başlangıç zamanından o anki zaman çıkarılarak toplam süre bulunur.
6. Çözülen matris ekrana basılır, toplam geçen süre ve algoritmanın toplam yaptığı geri alma sayısı ekrana yazılır.
7. Skor hesaplamak için lol_calculate_score fonksiyonu çağırılır ve skor nesnesi oluşturulur.
8. Bu nesne bağlı listeye Alfabetik ve puan sırasına göre ekleme yapan scorelist_add_in_order fonksiyonu ile Tüm oyunların skor ve kullanıcı bilgisi olan listeye yazılır. Bu liste de dosyaya yazdırılır, ve kullanıcıya algoritmanın aldığı puan bilgisi verilir.
9. Last.dat dosyası, çözülmüş matrisin bir daha yüklenmesini engellemek için silinir.

```

/* Input: GameState *g.    Output: bool (success/fail), GameState *g */
/* solve given matrix (g->mat[g->matrixCount-1]) */
bool gamestate_solver(GameState *g)
{
    PosList startPosLst = {0}, endPosLst = {0};
    Matrix *mat = NULL;
    bool retVal = true;

    CHECK_GAMESTATE(g, return false);
    if(g->matrixCount == 0)
    {
        fprintf(stderr, "There is no matrix to solve\n");
        return false;
    }

    CHECK_OPERATION(gamestate_top_matrix(g, &mat), return false);
    CHECK_OPERATION(poslist_init(&startPosLst, mat->N), return false);
    CHECK_OPERATION(poslist_init(&endPosLst, mat->N), poslist_deinit(&startPosLst); return false);

    if(matrix_to_poslist(mat, &startPosLst, &endPosLst) == false)
    {
        poslist_deinit(&startPosLst);
        poslist_deinit(&endPosLst);
        return false;
    }

    #if (VERBOSE_LEVEL >= 2)
    {int i = 0;
    for(i = 0; i < startPosLst.N; i++)
    {
        printf("startPosLst.pos[%d] = {%d, %d}\n", i, startPosLst.pos[i].X, startPosLst.pos[i].Y);
        printf("endPosLst.pos[%d] = {%d, %d}\n", i, endPosLst.pos[i].X, endPosLst.pos[i].Y);
    }
    printf("\n");}
    #endif

    retVal = gamestate_recursive_solver(g, &startPosLst, &endPosLst, 0);

    poslist_deinit(&startPosLst);
    poslist_deinit(&endPosLst);

    return retVal;
}

```

game_menu_automatic fonksiyonundan çağırılan, verilen matrisi çözmeyi sağlayan gamestate_solver fonksiyonunun adım adım açıklaması:

1. Önce CHECK_GAMESTATE makrosu ile gelen parametre kontrol edilir, ardından matrisin varlığı kontrol edilir.
2. Matris bulundu ise mat değişkenine listenin en üstündeki matris bilgisi verilir.
3. Algoritmada kullanılacak, sayıların başlangıç ve bitiş noktalarının listesi başlatılır ve bu noktalar hesaplanır.
4. gamestate_recursive_solver fonksiyonu (aşağıda açıklanmıştır) ile matris çözülmeye çalışılır..
5. Oluşturulan PosList yapıları yok edilir.
6. Matrisin çözülüp çözülmemeye bilgisini tutan retVal değişkeni döndürülür.

gamestate_recursive_solver

```
/* Input: GameState *g, PosList *currPosLst, const PosList *endPosLst, int varIndex.   Output: bool (success/fail), GameState *g, PosList *currPosLst */
/* internal function for solver */
bool gamestate_recursive_solver(GameState *g, PosList *currPosLst, const PosList *endPosLst, int varIndex)
{
    int i = 0;
    Matrix *mat = &g->mat[g->matrixCount-1];

    /* parameter will not check because cost of check in every iteration is high */

    if(varIndex >= mat->N) /* finished */
    {
        return matrix_is_solved(mat, currPosLst, endPosLst); /* check is solved */
    }

    if(currPosLst->pos[varIndex].X == endPosLst->pos[varIndex].X && currPosLst->pos[varIndex].Y == endPosLst->pos[varIndex].Y)
    {
        return gamestate_recursive_solver(g, currPosLst, endPosLst, varIndex+1);
    }

    if(matrix_has_dead_end(mat, currPosLst, endPosLst) == true)
    {
        return false;
    }

    for(i = 0; i < DIRECTION_COUNT; i++)
    {
        Position oldPos = currPosLst->pos[varIndex];
        Position newPos = {0, 0};

        NEXT_POSITION(i, &newPos, &oldPos);

        if(position_is_in_range(newPos, mat->N))
        {
            if( mat->data[newPos.Y][newPos.X] == MATRIX_EMPTY_ELEMENT ||
                (endPosLst->pos[varIndex].X == newPos.X &&
                 endPosLst->pos[varIndex].Y == newPos.Y)
            )
            {
                int oldVal = mat->data[newPos.Y][newPos.X];
                currPosLst->pos[varIndex] = newPos;
                mat->data[newPos.Y][newPos.X] = varIndex+1;

                #if (SAVE_GAMESTATE_IN_AUTO_MODE == 1)
                gamestate_file_write(g);
                #endif

                #if (PRINT_MOVE_UNDO_IN_AUTO_MODE == 1)
                printf("moving number %d from (%d, %d) to (%d, %d)\n", varIndex+1, oldPos.Y, oldPos.X, newPos.Y, newPos.X);
                #endif

                if(gamestate_recursive_solver(g, currPosLst, endPosLst, varIndex) == true)
                {
                    return true;
                }

                #if (PRINT_MOVE_UNDO_IN_AUTO_MODE == 1)
                printf("undoing %d from (%d, %d)\n", varIndex+1, newPos.Y, newPos.X);
                #endif

                g->undoCount++;

                currPosLst->pos[varIndex] = oldPos;
                mat->data[newPos.Y][newPos.X] = oldVal;

                #if (SAVE_GAMESTATE_IN_AUTO_MODE == 1)
                gamestate_file_write(g);
                #endif
            }
        }
    }

    return false;
}
```

```
    #if (SAVE_GAMESTATE_IN_AUTO_MODE == 1)
    gamestate_file_write(g);
    #endif

    #if (PRINT_MOVE_UNDO_IN_AUTO_MODE == 1)
    printf("moving number %d from (%d, %d) to (%d, %d)\n", varIndex+1, oldPos.Y, oldPos.X, newPos.Y, newPos.X);
    #endif

    if(gamestate_recursive_solver(g, currPosLst, endPosLst, varIndex) == true)
    {
        return true;
    }

    #if (PRINT_MOVE_UNDO_IN_AUTO_MODE == 1)
    printf("undoing %d from (%d, %d)\n", varIndex+1, newPos.Y, newPos.X);
    #endif

    g->undoCount++;

    currPosLst->pos[varIndex] = oldPos;
    mat->data[newPos.Y][newPos.X] = oldVal;

    #if (SAVE_GAMESTATE_IN_AUTO_MODE == 1)
    gamestate_file_write(g);
    #endif
}

return false;
}
```

gamestate_solver fonksiyonu tarafından çağırılan ve matrisin çözümünü rekürsif bir biçimde gerçekleştiren gamestate_recursive_solver fonksiyonunun adım adım açıklaması:

1. Önce 1. base-case olarak değişken sırası matrisin boyutundan büyükse, diğer bir değişle tüm sayılar gidebildiği kadar yolu gidebildiyse matrisin çözülmüş olup olmadığını kontrol eder ve bu değeri döner.
2. Sonra 2. base-case olarak şu anda bulunulan konumun hedef olup olmadığını kontrol eder. Eğer hedefin üzerinde ise bu sayıdan daha fazla ilerlenmemesi ve bir sonraki sayıya geçilmesi gerekir, bir sonraki sayıya geçerek kendisini bir sonraki indis ile çağırır ve bu çağırının dönüş değerini döner.
3. Son base-case olarak da matris üzerinde herhangi bir çıkmaz sokak bulunup bulunmadığını kontrol eder, Çıkmaz sokak, kendisine komşu boş kutuların ve kuyruk (currPosLst veya endPosLst) sayısının toplamı 2'den az olan boş kutulara denir. Eğer çıkmaz sokak oluştu ise buraya giren bir sayının hedefine ulaşma imkanı yok demektir. Ancak boş yer kalmaması gerektiği için bu boş kutunun da bir şekilde dolması gerekmektedir. İşte bu çelişkiden dolayı eğer bir çıkmaz sokak var ise matrisin çözülemeyen olduğunu doğrudan söyleyebiliriz.
4. Base-case kontrollerinden geçtikten sonra algoritma şu anda bulunulan yerin (currPosLst->pos[varIndex]) 4 tarafına sırayla şunları yapar:
5. Sıradaki konum alınır, bu konumun sınırlar içerisinde olup olmadığını kontrol eder, eğer sınırlar içerisinde ise o noktanın boş veya indis için bitiş noktası olup olmadığını kontrol eder.
6. Eğer 5. Adımdaki şartlar sağlanıyorsa algoritma o yöne sapar ve o noktaya indisdeki sayıyı yazar.
7. Tanımlardan aktif edildi ise bu değişikliği dosyaya ve ekrana yazar.
8. Algoritma aynı indis ile kendisini bir daha çağırır ve bir sonraki adım için 1. Maddeden başlayarak aynı işlemleri uygular.
9. Rekürsif çağrılarının birinde base-case'lerden birine takılıp fonksiyon geriye dönmeye başlar.
10. Bu dönüş işleminde eğer base-case durumunda matrisin çözüldüğü bilgisi varsa yukarıya doğru true bilgisi taşınır ve en tepedeki çağırardan da true değeri gamestate_solver fonksiyonuna döner.
11. Eğer base-case'ler bu matrisin bu yoldan çözülemeyeceği bilgisini taşırsa en dipteki fonksiyondan itibaren 6. Adımda yapılan o noktaya yazma işlemini geri alır ve geri dönüş miktarı değerini bir artırır.
12. Tanımlarda aktif edildi ise bu değişiklik de dosyaya ve ekrana yazılır.
13. Matrisin o noktasından hiçbir yöne doğru çözüm yoksa bir üst çağrıya false değeri döndürülür ve bir üst çağırının da başka bir yön denemesi sağlanır.
14. En tepedeki çağrıya kadar herhangi bir çözüm yolu bulunamadıysa gamestate_solver fonksiyonuna false değeri döndürülür.

Yol bulma örneği

```
      | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4|  |  |  | 3|
-----

+-----+
| 1- Play in Automatic Mode |
| 2- Play in Manual Mode   |
| 3- Return to Main Menu    |
+-----+
Enter game menu operation [1-3]: 1
```



```

moving number 1 from (1, 1) to (1, 0)
moving number 1 from (1, 0) to (0, 0)
moving number 1 from (0, 0) to (0, 1)
moving number 1 from (0, 1) to (0, 2)
moving number 1 from (0, 2) to (1, 2)
moving number 1 from (1, 2) to (1, 3)
moving number 1 from (1, 3) to (1, 4)
undoing 1 from (1, 4)
undoing 1 from (1, 3)
moving number 1 from (1, 2) to (2, 2)
moving number 1 from (2, 2) to (2, 1)
moving number 1 from (2, 1) to (2, 0)
undoing 1 from (2, 0)
moving number 1 from (2, 1) to (3, 1)
undoing 1 from (3, 1)
undoing 1 from (2, 1)
undoing 1 from (2, 2)
undoing 1 from (1, 2)
undoing 1 from (0, 2)
undoing 1 from (0, 1)
undoing 1 from (0, 0)
moving number 1 from (1, 0) to (2, 0)
undoing 1 from (2, 0)
undoing 1 from (1, 0)
moving number 1 from (1, 1) to (1, 2)
moving number 1 from (1, 2) to (1, 3)
moving number 1 from (1, 3) to (1, 4)
undoing 1 from (1, 4)
undoing 1 from (1, 3)
moving number 1 from (1, 2) to (0, 2)
moving number 1 from (0, 2) to (0, 1)
moving number 1 from (0, 1) to (0, 0)
moving number 1 from (0, 0) to (1, 0)
moving number 1 from (1, 0) to (2, 0)
moving number 1 from (2, 0) to (2, 1)
moving number 1 from (2, 1) to (2, 2)
undoing 1 from (2, 2)
moving number 1 from (2, 1) to (3, 1)
moving number 2 from (0, 3) to (1, 3)
moving number 2 from (1, 3) to (1, 4)
undoing 2 from (1, 4)
undoing 2 from (1, 3)
undoing 1 from (3, 1)
undoing 1 from (2, 1)
undoing 1 from (2, 0)
undoing 1 from (1, 0)
undoing 1 from (0, 0)
undoing 1 from (0, 1)
undoing 1 from (0, 2)
moving number 1 from (1, 2) to (2, 2)
moving number 1 from (2, 2) to (2, 1)
moving number 1 from (2, 1) to (2, 0)
moving number 1 from (2, 0) to (1, 0)
moving number 1 from (1, 0) to (0, 0)
moving number 1 from (0, 0) to (0, 1)
moving number 1 from (0, 1) to (0, 2)
undoing 1 from (0, 2)
undoing 1 from (0, 1)
undoing 1 from (0, 0)
undoing 1 from (1, 0)
undoing 1 from (2, 0)
moving number 1 from (2, 1) to (3, 1)
moving number 2 from (0, 3) to (0, 2)
moving number 2 from (0, 2) to (0, 1)
moving number 2 from (0, 1) to (0, 0)
moving number 2 from (0, 0) to (1, 0)

```



```

undoing 1 from (3, 1)
undoing 1 from (2, 1)
undoing 1 from (2, 0)
undoing 1 from (1, 0)
undoing 1 from (0, 0)
moving number 1 from (0, 1) to (0, 2)
undoing 1 from (0, 2)
undoing 1 from (0, 1)
moving number 1 from (1, 1) to (2, 1)
moving number 1 from (2, 1) to (2, 0)
moving number 1 from (2, 0) to (1, 0)
moving number 1 from (1, 0) to (0, 0)
moving number 1 from (0, 0) to (0, 1)
moving number 1 from (0, 1) to (0, 2)
moving number 1 from (0, 2) to (1, 2)
moving number 1 from (1, 2) to (1, 3)
moving number 1 from (1, 3) to (1, 4)
undoing 1 from (1, 4)
undoing 1 from (1, 3)
moving number 1 from (1, 2) to (2, 2)
undoing 1 from (2, 2)
undoing 1 from (1, 2)
undoing 1 from (0, 2)
undoing 1 from (0, 1)
undoing 1 from (0, 0)
undoing 1 from (1, 0)
undoing 1 from (2, 0)
moving number 1 from (2, 1) to (2, 2)
moving number 1 from (2, 2) to (1, 2)
moving number 1 from (1, 2) to (1, 3)
moving number 1 from (1, 3) to (1, 4)
undoing 1 from (1, 4)
undoing 1 from (1, 3)
moving number 1 from (1, 2) to (0, 2)
moving number 1 from (0, 2) to (0, 1)
moving number 1 from (0, 1) to (0, 0)
moving number 1 from (0, 0) to (1, 0)
moving number 1 from (1, 0) to (2, 0)
undoing 1 from (2, 0)
undoing 1 from (1, 0)
undoing 1 from (0, 0)
undoing 1 from (0, 1)
undoing 1 from (0, 2)
undoing 1 from (1, 2)
undoing 1 from (2, 2)
moving number 1 from (2, 1) to (3, 1)
moving number 2 from (0, 3) to (0, 2)
moving number 2 from (0, 2) to (0, 1)
moving number 2 from (0, 1) to (0, 0)
moving number 2 from (0, 0) to (1, 0)
moving number 2 from (1, 0) to (2, 0)
moving number 2 from (2, 0) to (3, 0)
moving number 3 from (2, 4) to (1, 4)
moving number 3 from (1, 4) to (1, 3)
moving number 3 from (1, 3) to (1, 2)
moving number 3 from (1, 2) to (2, 2)
undoing 3 from (2, 2)
undoing 3 from (1, 2)
undoing 3 from (1, 3)
undoing 3 from (1, 4)
moving number 3 from (2, 4) to (3, 4)
moving number 3 from (3, 4) to (3, 3)
moving number 3 from (3, 3) to (4, 3)
moving number 3 from (4, 3) to (4, 2)
moving number 3 from (4, 2) to (4, 1)

```

```

undoing 3 from (4, 1)
undoing 3 from (4, 2)
moving number 3 from (4, 3) to (4, 4)
moving number 4 from (2, 3) to (2, 2)
moving number 4 from (2, 2) to (1, 2)
moving number 4 from (1, 2) to (1, 3)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 4 from (1, 2)
undoing 4 from (2, 2)
moving number 4 from (2, 3) to (1, 3)
moving number 4 from (1, 3) to (1, 2)
undoing 4 from (1, 2)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 3 from (4, 4)
undoing 3 from (4, 3)
undoing 3 from (3, 3)
moving number 3 from (3, 4) to (4, 4)
moving number 4 from (2, 3) to (2, 2)
moving number 4 from (2, 2) to (1, 2)
moving number 4 from (1, 2) to (1, 3)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
undoing 4 from (1, 2)
undoing 4 from (2, 2)
moving number 4 from (2, 3) to (1, 3)
moving number 4 from (1, 3) to (1, 2)
undoing 4 from (1, 2)
moving number 4 from (1, 3) to (1, 4)
undoing 4 from (1, 4)
undoing 4 from (1, 3)
moving number 4 from (2, 3) to (3, 3)
moving number 4 from (3, 3) to (4, 3)
moving number 4 from (4, 3) to (4, 2)
moving number 4 from (4, 2) to (4, 1)
moving number 4 from (4, 1) to (4, 0)
moving number 5 from (0, 4) to (1, 4)
moving number 5 from (1, 4) to (1, 3)
moving number 5 from (1, 3) to (1, 2)
moving number 5 from (1, 2) to (2, 2)
moving number 5 from (2, 2) to (3, 2)

```

```

| 0| 1| 2| 3| 4|
-----
0|| 2| 2| 2| 2| 5|
-----
1|| 2| 1| 5| 5| 5|
-----
2|| 2| 1| 5| 4| 3|
-----
3|| 2| 1| 5| 4| 3|
-----
4|| 4| 4| 4| 4| 3|
-----

```

```

Time(second) elapsed: 0
Undo count: 112
Score: 872.066511

```

Manuel Modda Yol Bulma

Yol bulma kodu ve açıklamaları

game_menu_manual

```
/* Input: ListOfLists *lst.   Output: bool (success/fail), ListOfLists *lst */
/* manual gameplay menu. return true if succeed */
bool game_menu_manual(ListOfLists *lst)
{
    bool (* const inGameMenuFp[INGAME_MENU_COUNT]) (ListOfLists *lst) = {
        ingame_menu_undo,
        ingame_menu_move
    };
    InGameMenuOperations op = 0;

    CHECK_LISTOFLISTS(lst, return false);

    if(lst->g.matrixCount == 0)
    {
        fprintf(stderr, "There is no matrix to solve\n");
        return false;
    }

    lst->g.totalTime = (lst->g.totalTime == 0 ? (unsigned long)time(NULL) : lst->g.totalTime);

    gamestate_file_write(&lst->g);

    do
    {
        matrix_print(&lst->g.mat[lst->g.matrixCount-1]);
        ingame_menu_print();
        op = ingame_menu_get_operation();

        if(op < INGAME_MENU_RESIGN)
        {
            inGameMenuFp[op](lst);
        }
    }while(op != INGAME_MENU_RESIGN && matrix_is_solved(&lst->g.mat[lst->g.matrixCount-1], NULL, NULL) == false);

    if(op != INGAME_MENU_RESIGN)
    {
        Score *s = NULL;
        matrix_print(&lst->g.mat[lst->g.matrixCount-1]);

        lst->g.totalTime = time(NULL) - lst->g.totalTime;

        printf("Time (second) elapsed: %lu\n", lst->g.totalTime);
        printf("Undo count: %lu\n", lst->g.undoCount);

        s = score_create(lst->g.playerName, lol_calculate_score(lst));
        scorelist_add_in_order(&lst->s1st, s);
        scorelist_file_write(&lst->s1st);
        printf("Score: %lf\n", s->score);
    }
    else
    {
        printf("Resigned\n");
    }
    remove(FILENAME_LAST_GAME);

    return true;
}
```

Matris dosyadan alındıktan veya rastgele oluşturulduktan sonraki menüde manuel çözmeye seçildiğinde game_menu_manual fonksiyonu çağırılmaktadır. Fonksiyonun adım adım açıklaması:

1. Önce CHECK_LISTOFLISTS makrosu ile gelen parametrenin doğruluğu ve hemen ardından GameState içinde herhangi bir matris olup olmadığı kontrol edilir.

2. Oyunun başlangıcından sona kadar geçen süreyi ölçebilmek için işleme başlamadan önce oyun bir kayıttan yüklenmediyse o anki zamanı time fonksiyonu ile alır.
3. Kullanıcı oyuna henüz başlamadan gamestate_file_write ile oyunun başlangıç durumu dosyaya yazılır.
4. Kullanıcı matris çözülmeye veya kullanıcı oyunu terk edene kadar 2 farklı işlemten birini yapabilir: 1. geri alma, 2. hareket yapma (aşağıda detaylandırılmıştır)
5. Oyun matris çözülmüş bir şekilde sonlandıysa toplam süre, o anki zamandan başlangıçtaki zaman çıkarılarak bulunur, sonrasında toplam süre ve geri alma sayısı ekrana yazılır ardından skor hesaplanıp ekrana ve dosyaya yazılır.
6. Oyun terk etme biçiminde sonlandıysa ekrana bununla ilgili yazı yazılır.
7. En sonunda Last.dat dosyası bir sonraki yüklemde bitmiş veya terk edilmiş bir oyun yüklenmemesi için silinir.

ingame_menu_undo

```
/* Input: ListOfLists *lst.    Output: bool (success/fail), ListOfLists *lst */
/* undo last movement. return true if succeed */
bool ingame_menu_undo(ListOfLists *lst)
{
    CHECK_LISTOFLISTS(lst, return false);

    if(lst->g.matrixCount <= 1)
    {
        fprintf(stderr, "Undo stack is empty\n");
        return false;
    }

    lst->g.undoCount++;

    CHECK_OPERATION(gamestate_pop_matrix(&lst->g), return false);
    CHECK_OPERATION(gamestate_file_pop(&lst->g), return false);

    return true;
}
```

game_menu_manual fonksiyonunun içinde kullanıcının yapabileceği işlemlerden biri geri alma işlemidir, bu işlem ingame_menu_undo fonksiyonu ile gerçekleştirilir. Fonksiyonun detaylı açıklaması:

1. Verilen parametreler kontrol edilir.
2. O anda matris sayısı 1 veya daha az ise matrisin geri alınamayacağı ekrana yazılıp false değeri döndürülür.
3. gamestate_pop_matrix fonksiyonu ile matris listesinden en sondaki değer silinir.
4. gamestate_file_pop fonksiyonu ile son matrisi silinmiş GameState yapısı dosyaya yazdırılır.
5. undoCount 1 artırılır ve fonksiyondan true döndülür.

```

/* Input: ListOfLists *lst.   Output: bool (success/fail), ListOfLists *lst */
/* movement. return true if succeed */
bool ingame_menu_move(ListOfLists *lst)
{
    Position srcPos = {0}, destPos = {0};
    Matrix mat = {0};
    int i = 0;

    CHECK_LISTOFLISTS(lst, return false);

    if(lst->g.matrixCount == 0)
    {
        fprintf(stderr, "There is not matrix\n");
        return false;
    }

    CHECK_OPERATION(matrix_duplicate(&lst->g.mat[lst->g.matrixCount-1], &mat), return false);

    srcPos = get_position_input(0, 0, lst->g.matrixSize-1, lst->g.matrixSize-1, "Enter source position (row, column): ");
    destPos = get_position_input(0, 0, lst->g.matrixSize-1, lst->g.matrixSize-1, "Enter destination position (row, column): ");

    if(position_is_aligned(srcPos, destPos) == false)
    {
        fprintf(stderr, "Coordinates are not aligned\n");
        matrix_deinit(&mat);
        return false;
    }
    else if(position_is_same(srcPos, destPos))
    {
        fprintf(stderr, "Coordinates are same\n");
        matrix_deinit(&mat);
        return false;
    }

    if(mat.data[srcPos.Y][srcPos.X] == MATRIX_EMPTY_ELEMENT)
    {
        fprintf(stderr, "Position (%d, %d) is empty\n", srcPos.Y, srcPos.X);
        matrix_deinit(&mat);
        return false;
    }

    if(srcPos.X == destPos.X)
    {
        bool isValid = true;
        const int from = MIN(srcPos.Y, destPos.Y);
        const int to = MAX(srcPos.Y, destPos.Y);

        for(i = from; i <= to && isValid == true; i++)
        {
            if(mat.data[srcPos.Y][srcPos.X] != mat.data[i][srcPos.X] &&
               mat.data[i][srcPos.X] != MATRIX_EMPTY_ELEMENT
            )
            {
                isValid = false;
            }
        }
        if(isValid == false)
        {
            fprintf(stderr, "Line is not empty or not same element as source coordinate element\n");
            matrix_deinit(&mat);
            return false;
        }

        /* make movement */
        for(i = from; i <= to; i++)
        {
            mat.data[i][srcPos.X] = mat.data[srcPos.Y][srcPos.X];
        }
    }
}

```



```

else if(srcPos.Y == destPos.Y)
{
    bool isValid = true;
    const int from = MIN(srcPos.X, destPos.X);
    const int to = MAX(srcPos.X, destPos.X);

    for(i = from; i <= to && isValid == true; i++)
    {
        if(mat.data[srcPos.Y][srcPos.X] != mat.data[srcPos.Y][i] &&
           mat.data[srcPos.Y][i] != MATRIX_EMPTY_ELEMENT
        )
        {
            isValid = false;
        }
    }
    if(isValid == false)
    {
        fprintf(stderr, "Line is not empty or not same element as srcPos coordinate element\n");
        matrix_deinit(&mat);
        return false;
    }

    /* make movement */
    for(i = from; i <= to; i++)
    {
        mat.data[srcPos.Y][i] = mat.data[srcPos.Y][srcPos.X];
    }
}

if(gamestate_push_matrix(&lst->g, &mat) == false)
{
    matrix_deinit(&mat);
    return false;
}
return gamestate_file_push(&lst->g);
}

```

game_menu_manual fonksiyonunun içinde kullanıcının yapabileceği işlemlerden bir diğeri de hareket yapma işlemidir, bu işlem ingame_menu_move fonksiyonu ile gerçekleştirilir. Fonksiyonun detaylı açıklaması:

1. Parametre kontrolleri yapılır, matris listesindeki matris sayısı 0 ise false dönülür.
2. Matris listesindeki son matris klonlanır, ve bu klonlanan matris üzerinden kalan işlemler yapılır.
3. Kullanıcıya hangi koordinattan hangi koordinata hareket işlemini gerçekleştirmek istediği sorulur.
4. Girilen koordinatlar aynı hizada değilse, birebir aynı koordinatsa veya kaynak koordinat boşsa klonlanan matris silinip false değeri dönülür.
5. Eğer kaynak ve varış noktasının X (sütun) değerleri aynıysa veya Y (satır) değerleri aynıysa aynı algoritmanın sadece indis sırası değişmiş hali çalışacaktır. Bu algoritma şöyledir:
6. Küçük olan koordinattan büyük olan koordinata doğru tüm kutular boş veya kaynak ile aynı değere sahip olup olmadığına göre kontrol edilir.
7. Eğer yol üzerinde geçersiz bir eleman varsa kullanıcıya hata mesajı gösterilip false değeri döndürülür.
8. Yol geçerli ise yol üzerindeki koordinatların hepsi kaynak koordinatın değeri yapılır.
9. Üzerinde işlem yapılan (en başta klonlanan) matris listesinin sonuna eklenir.
10. Yeni eklenen matris listesi ve yeni matris sayısı dosyaya (Last.dat) yazılır.

Yol bulma örneği

```

+-----+
| 1- Create Random Matrix |
| 2- Create Matrix from File |
| 3- Show User Scores |
| 4- Exit |
+-----+
Enter main menu operation [1-4]: 2
Enter matrix size [3-10]: 5
Enter Player Name: Alperen
Enter 5x5 matrix file name: ornek.txt

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4|  |  |  | 3|
-----

+-----+
| 1- Play in Automatic Mode |
| 2- Play in Manual Mode |
| 3- Return to Main Menu |
+-----+
Enter game menu operation [1-3]: 2

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4|  |  |  | 3|
-----

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+
Enter operation [1-3]: |

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4|  |  |  | 3|
-----

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+
Enter operation [1-3]: |

```

```

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4|  |  |  | 3|
-----

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 4 0
Enter destination position (row, column): 4 3

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  |  |  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4| 4| 4| 4| 3|
-----

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 1 1
Enter destination position (row, column): 3 1

  | 0| 1| 2| 3| 4|
-----
0||  |  |  | 2| 5|
-----
1||  | 1|  |  |  |
-----
2||  | 1|  | 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4| 4| 4| 4| 3|
-----

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+
Enter operation [1-3]: 2
Enter source position (row, column): 3 2
Enter destination position (row, column): 0 2

  | 0| 1| 2| 3| 4|
-----
0||  |  | 5| 2| 5|
-----
1||  | 1| 5|  |  |
-----
2||  | 1| 5| 4| 3|
-----
3|| 2| 1| 5|  |  |
-----
4|| 4| 4| 4| 4| 3|
-----

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 1

```

      | 0| 1| 2| 3| 4|
+-----+
0||  |  |  | 2| 5|
+-----+
1||  | 1|  |  |  |
+-----+
2||  | 1|  | 4| 3|
+-----+
3|| 2| 1| 5|  |  |
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 3 2

Enter destination position (row, column): 1 2

```

      | 0| 1| 2| 3| 4|
+-----+
0||  |  |  | 2| 5|
+-----+
1||  | 1| 5|  |  |
+-----+
2||  | 1| 5| 4| 3|
+-----+
3|| 2| 1| 5|  |  |
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 1 2

Enter destination position (row, column): 1 4

```

      | 0| 1| 2| 3| 4|
+-----+
0||  |  |  | 2| 5|
+-----+
1||  | 1| 5| 5| 5|
+-----+
2||  | 1| 5| 4| 3|
+-----+
3|| 2| 1| 5|  |  |
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 4 3

Enter destination position (row, column): 2 3

```

      | 0| 1| 2| 3| 4|
+-----+
0||  |  |  | 2| 5|
+-----+
1||  | 1| 5| 5| 5|
+-----+
2||  | 1| 5| 4| 3|
+-----+
3|| 2| 1| 5| 4|  |
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 4 4

Enter destination position (row, column): 2 4

```

      | 0| 1| 2| 3| 4|
+-----+
0||  |  |  | 2| 5|
+-----+
1||  | 1| 5| 5| 5|
+-----+
2||  | 1| 5| 4| 3|
+-----+
3|| 2| 1| 5| 4| 3|
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 3 0

Enter destination position (row, column): 0 0

```

      | 0| 1| 2| 3| 4|
+-----+
0|| 2|  |  | 2| 5|
+-----+
1|| 2| 1| 5| 5| 5|
+-----+
2|| 2| 1| 5| 4| 3|
+-----+
3|| 2| 1| 5| 4| 3|
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

```

+-----+
| 1- Undo |
| 2- Move |
| 3- Resign |
+-----+

```

Enter operation [1-3]: 2

Enter source position (row, column): 0 0

Enter destination position (row, column): 0 3

```

      | 0| 1| 2| 3| 4|
+-----+
0|| 2| 2| 2| 2| 5|
+-----+
1|| 2| 1| 5| 5| 5|
+-----+
2|| 2| 1| 5| 4| 3|
+-----+
3|| 2| 1| 5| 4| 3|
+-----+
4|| 4| 4| 4| 4| 3|
+-----+

```

Time (second) elapsed: 78

Undo count: 1

Score: 2863.182210

Oyun ve Skor İçin Tasarlanan Yapılar

Oyun için gerekli bazı yapılar (enum, struct) tasarlanmıştır.

Enum

```
typedef enum MainMenuOperations
{
    MAIN_MENU_RANDOM,
    MAIN_MENU_FILE,
    MAIN_MENU_SCORES,
    MAIN_MENU_EXIT
}MainMenuOperations;

typedef enum GameMenuOperations
{
    GAME_MENU_AUTOMATIC,
    GAME_MENU_MAUNAL,
    GAME_MENU_RETURN
}GameMenuOperations;

typedef enum InGameMenuOperations
{
    INGAME_MENU_UNDO,
    INGAME_MENU_MOVE,
    INGAME_MENU_RESIGN
}InGameMenuOperations;
```

- **MainMenuOperations:** Ana menüde kullanıcının seçebileceği işlemler.
- **GameMenuOperations:** Oyun menüsünde (matris oluşturulduktan veya alındıktan sonraki menü) kullanıcının seçebileceği işlemler.
- **InGameMenuOperations:** Oyun için menüsünde (manuel mod) kullanıcının seçebileceği işlemler.

Struct

```
typedef struct Position
{
    int X, Y;
}Position;

typedef struct PosList
{
    Position *pos; /* positions array */
    int N;         /* element count */
}PosList;
```

- **Position:** koordinat belirten yapı, X sütun, Y satır değerini belirtir.
- **PosList:** Position yapılarını tutan liste yapısı. N listedeki eleman sayısı, pos ise elemanların başladığı adresi tutar.

```
typedef struct Matrix
{
    int **data;
    int N;
}Matrix;
```

Matrix: Tam sayı değeri tutan matris yapısı. data matrisin satırlarını tutar, data[i] ise bir satırı belirtir, data[i][j] ise matrisin bir noktasındaki değerdir. N matrisin boyutunu belirtir (5 satır 5 sütun için N değeri 5 olur).

```
typedef struct GameState
{
    Matrix *mat; /* using as a stack */
    int matrixCount;

    int matrixSize; /* N */

    unsigned long totalTime; /* second */

    unsigned long undoCount;

    MainMenuOperations mainMenuOperations;

    GameMenuOperations gameMenuOperations;

    char playerName[LENGTH_PLAYER_NAME];
}GameState;
```

Gamestate: Oyunla ilgili gerekli çoğu bilginin tutulduğu yapı. GameState yapısının detaylı açıklaması:

- **Matrix *mat:** Matris dizisini tutar, bu yapı sayesinde geri alma işlemleri doğrudan bir önceki matrise geçerek yapılabilir.
- **int matrixCount:** Toplam matris sayısını tutar, mat dizisinin eleman sayısını belirtir.
- **int matrixSize:** Oyundaki matrisin boyutunu belirtir, mat[i]->N değeri ile aynı olur.
- **unsigned long totalTime:** Bitirilmiş oyunda oyunun başlangıç ve bitiş süresi arasındaki farkı tutar. Bitmemiş oyunda (Last.dat dosyasında) oyunun başlangıç zamanını tutar.
- **unsigned long undoCount:** Oyundaki toplam geri alma sayısını belirtir.
- **MainMenuOperations mainMenuOperations:** Ana menüden seçilen işlemi belirtir, skor hesabında kullanılır.
- **GameMenuOperations gameMenuOperations:** Matris oluşturulduktan veya dosyadan okunduktan sonraki menüde (Oyun menüsü) yapılan işlemin numarasını belirtir. Skor hesabında veya kaydedilmiş oyun dosyası (Last.dat) yüklenirken kullanılır.
- **char playerName[LENGTH_PLAYER_NAME]:** Skor hesabında kullanmak ve geçmişti tutabilmek için kullanıcıdan alınan isim.

```
typedef struct LastHeader
{
    char signature[LAST_HEADER_SIGNATURE_SIZE];
    int matrixCount;
    int matrixSize;
    unsigned long totalTime;
    unsigned long undoCount;
    MainMenuOperations mainMenuOperations;
    GameMenuOperations gameMenuOperations;
    char playerName[LENGTH_PLAYER_NAME];

    /* datas start followed by header */
}LastHeader;
```

LastHeader: oyun kayıt dosyası (Last.dat) dosyası oluşturulurken kullanılan dosyanın başlık bilgisidir. Bu başlık yapısı kayıt işlemi yapılmadan hemen önce GameState yapısından bu yapıya kopyalanır, ardından yapı doğrudan dosyanın başına yazılır. Dosyadan okuma yapılacağı zaman da önce başlık bu yapıya okunur ardından GameState yapısına çevirilir.

```

typedef struct Score
{
    char playerName[LENGTH_PLAYER_NAME];
    double score;
    struct Score *next;
}Score;

typedef struct ScoreList
{
    Score *head;
    int N;
}ScoreList;

```

- **Score:** Bağlı liste yapısında oyuncunun skor bilgisini tutan yapı. Oyuncu ismi gamestate'den kopyalanır, skor ise oyun bittiğinde hesaplanan değerdir.
- **ScoreList:** Score yapısını tutan sarmalayıcı yapıdır.

```

typedef struct ListOfLists
{
    GameState g;
    ScoreList slst;
}ListOfLists;

```

ListOfLists: Oyun için gerekli olan tüm yapıların tutulduğu sarmalayıcı yapısıdır.